ARIMA

Daniel de Freitas

Instituto J&F
Centro de Educação para os Negócios

Moving Average Model – MA(q)

Moving Average Model (MA(q)): definição

A ideia do MA(q) é modelar o valor atual como uma **combinação linear dos erros passados**, não dos valores passados (AR(p))

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

y_t = valor atual

μ = média da série

ε_t = ruído branco (erro atual)

θ_i = peso do erro i períodos atrás

q = quantidade de defasagens de erro usadas

🚨 🚨 **SMA e EMA já usam janelas móveis, mas a média móvel do MA(q) é sobre os erros, não sobre os valores** 🚨 🚨

Moving Average Model (MA(q)): definição

A ideia do MA(q) é modelar o valor atual como uma **combinação linear dos erros passados**, não dos valores passados (AR(p))

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

y_t = valor atual

μ = média da série

ε_t = ruído branco (erro atual)

θ_i = peso do erro i períodos atrás

q = quantidade de defasagens de erro usadas

MA(q) = ARIMA(0,0,q) no Python

ARMA(p,q)

ARMA: definição

O ARMA combina as duas ideias: ele tem uma parte auto-regressiva (AR) e uma parte de média móvel (MA) na mesma equação. **É adequado para séries estacionárias**

X_t: valor da série no tempo t

c: constante (média ajustada)

p: ordem do termo auto-regressivo (quantos lags de X são usados)

ϕ_i: coeficientes AR

ϵ_t: erro aleatório em t (ruído branco)

q: ordem do termo de média móvel

θ_j: coeficientes MA

$$X_t = c + \sum_{i=1}^{p} \phi_i X_{t-i} + \epsilon_t + \sum_{j=1}^{q} \theta_j \epsilon_{t-j}$$

ARIMA(p,d,q)

ARIMA: definição

O ARIMA é a generalização: ele acrescenta o **I (Integrated)**, que significa diferenciações aplicadas à série para remover a não-estacionariedade

$$\phi(L)(1-L)^d X_t = c + \theta(L)\epsilon_t$$

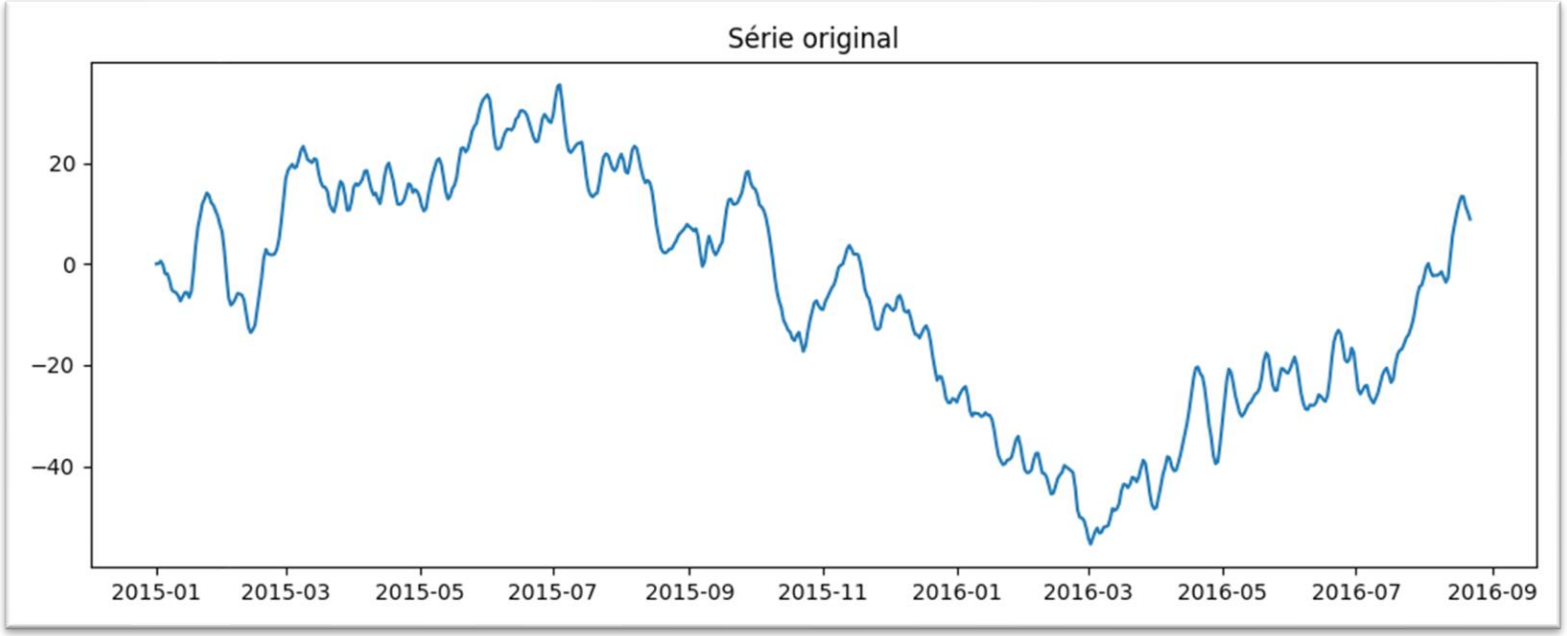
$\phi(L) = 1 - \phi_1 L - \phi_2 L^2 - \cdots - \phi_p L^p$ (parte AR)

$\theta(L) = 1 + \theta_1 L + \theta_2 L^2 + \cdots + \theta_q L^q$ (parte MA)

$(1-L)^d$: operador de diferenciação aplicado d vezes.

ARIMA: 1. serie

Realize o download do arquivo **serie_arima.xlsx** e plote a serie original:



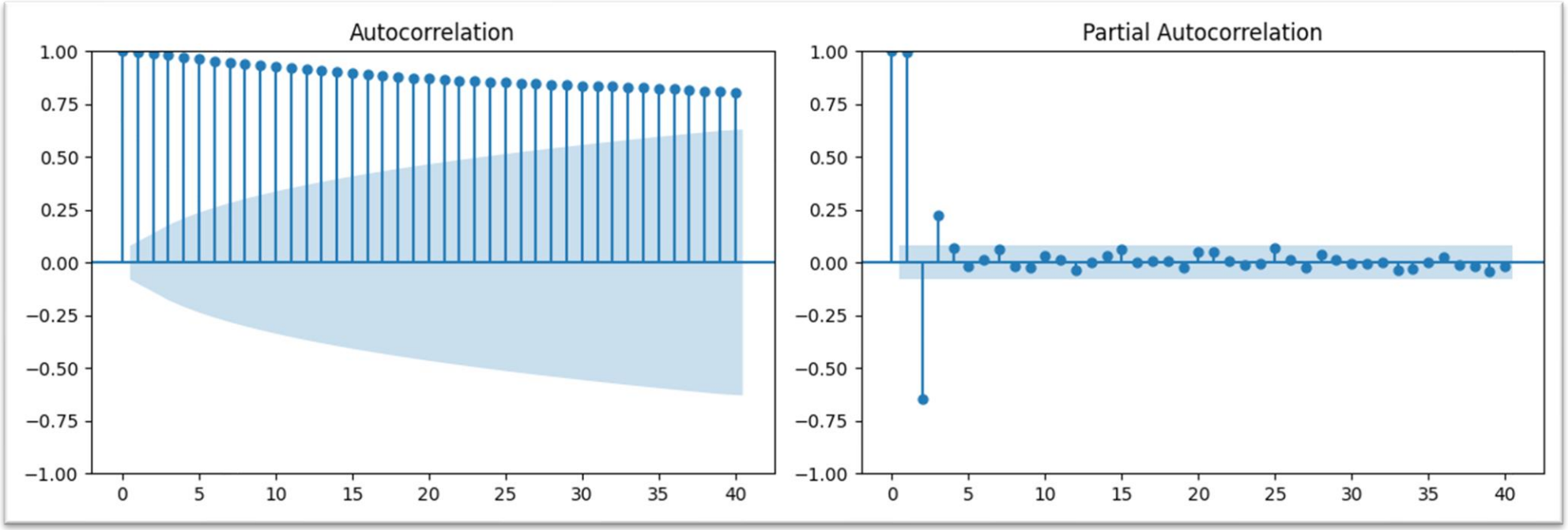
	y
2015-01-01	0.000000
2015-01-02	0.050000
2015-01-03	0.555962
2015-01-04	-0.186063
2015-01-05	-1.972122

ARIMA: 2. teste estacionariedade

Teste a estacionariedade e verifique o ACF e PACF:

ARIMA: 2. teste estacionariedade

Teste a estacionariedade e verifique o ACF e PACF:



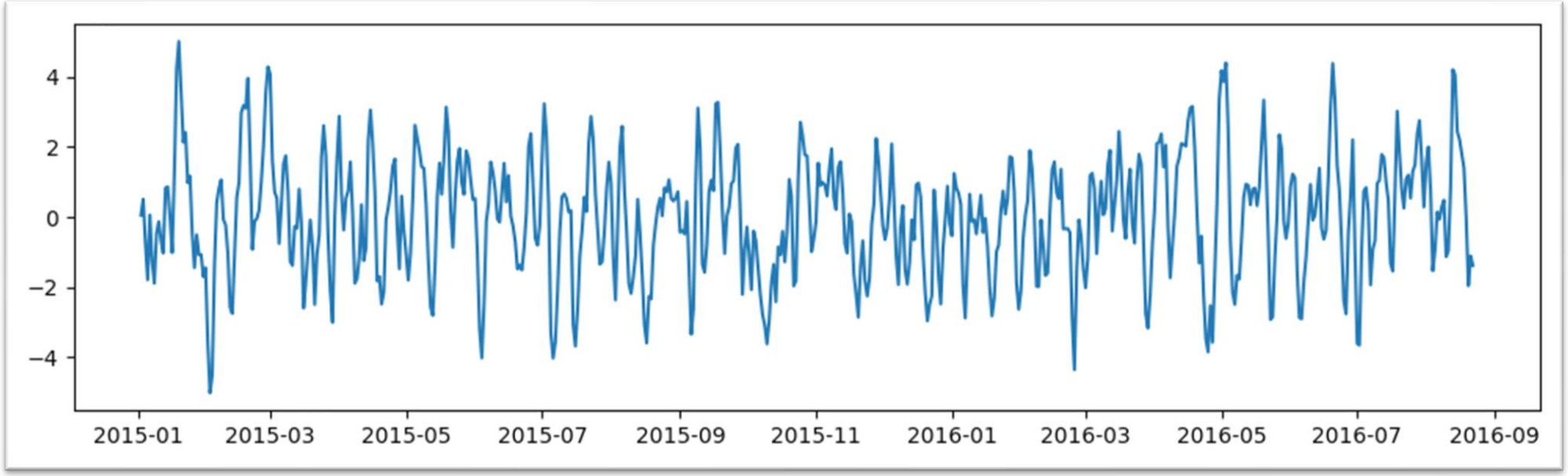

```
ADF p= 0.6064256234772181
KPSS p= 0.01
```

ARIMA: 3. diferenciação

Diferencie a serie, teste estacionariedade e plot ACF e PACF:

ARIMA: 3. diferenciação

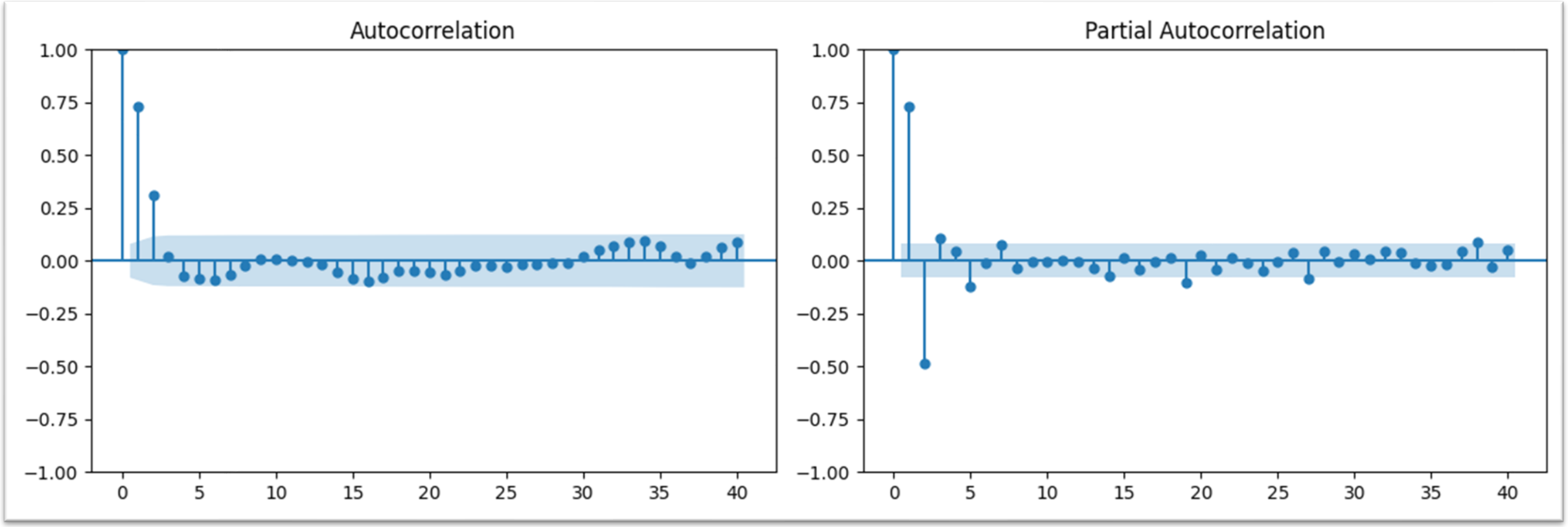
Diferencie a serie, teste estacionariedade e plot ACF e PACF:




```
ADF p= 1.9983908674457583e-18
KPSS p= 0.1
```

ARIMA: 3. diferenciação

Diferencie a serie, teste estacionariedade e plot ACF e PACF:



ARIMA: 4. treino - teste

Vamos realizar a separação treino- teste

```
# pensando em 3 diass para prever
h = 3
train = df["y"].iloc[:-h]
test  = df["y"].iloc[-h:]
```

Aqui definimos o horizonte de previsão h, ou seja, h dias que queremos que o modelo realize a previsão

- Quanto maior o horizonte de análise, maior a propagação do erro que irá ocorrer;

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"Melhor ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

- Devemos decidir o melhor modelo, ou seja, encontrar os **valores ideais para p,d e q**.
- Ignoramos o modelo trivial, p = 0, d = 0 e q = 0.
- Não colocamos i > 2 por boa pratica

```
Melhor ARIMA(0, 1, 3)  BIC=1715.06
```

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Suprimir os erros e importar o modelo ARIMA

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Variáveis auxiliares para guardarmos o menor BIC (melhor_bic), quais os parâmetros vencedores (melhor_ordem) e qual o modelo vencedor (melhor_modelo)

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Variando os parâmetros p, d e q. Lembre-se que o d não deve ser muito alto, algo padrão entre 0 e 2.

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Ignoramos o modelo trivial ARIMA(0,0,0)

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelo1  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Treinamos o modelo com os p,d e q parâmetros iterados no for. Obtemos o BIC desse modelo e armazenamos.

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Armazenamos as informações do primeiro modelo testado, ele será comparado com demais.

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo


print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

Armazenando informações do modelo que obtém o menor BIC.

ARIMA: 5. melhor modelo

```python
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tsa.arima.model import ARIMA


# variaveis auxiliaares
melhor_bic   = float("inf")
melhor_ordem = None
melhor_modelo   = None

for p in range(0, 5):
    for d in range(0, 3):        # 0 - 2 eh boa pratica
        for q in range(0, 5):
            if p == 0 and d == 0 and q == 0:
                continue  # nao considera o modelol  0 0 0
            modelo = ARIMA(train, order=(p, d, q)).fit()
            bic_for = float(modelo.bic)

            if melhor_ordem is None:
                melhor_bic   = bic_for
                melhor_ordem = (p, d, q)
                melhor_modelo   = modelo
            else:
                if bic_for < melhor_bic:
                    melhor_bic   = bic_for
                    melhor_ordem = (p, d, q)
                    melhor_modelo   = modelo
```

```python
print(f"best ARIMA{melhor_ordem}  BIC={melhor_bic:.2f}")
```

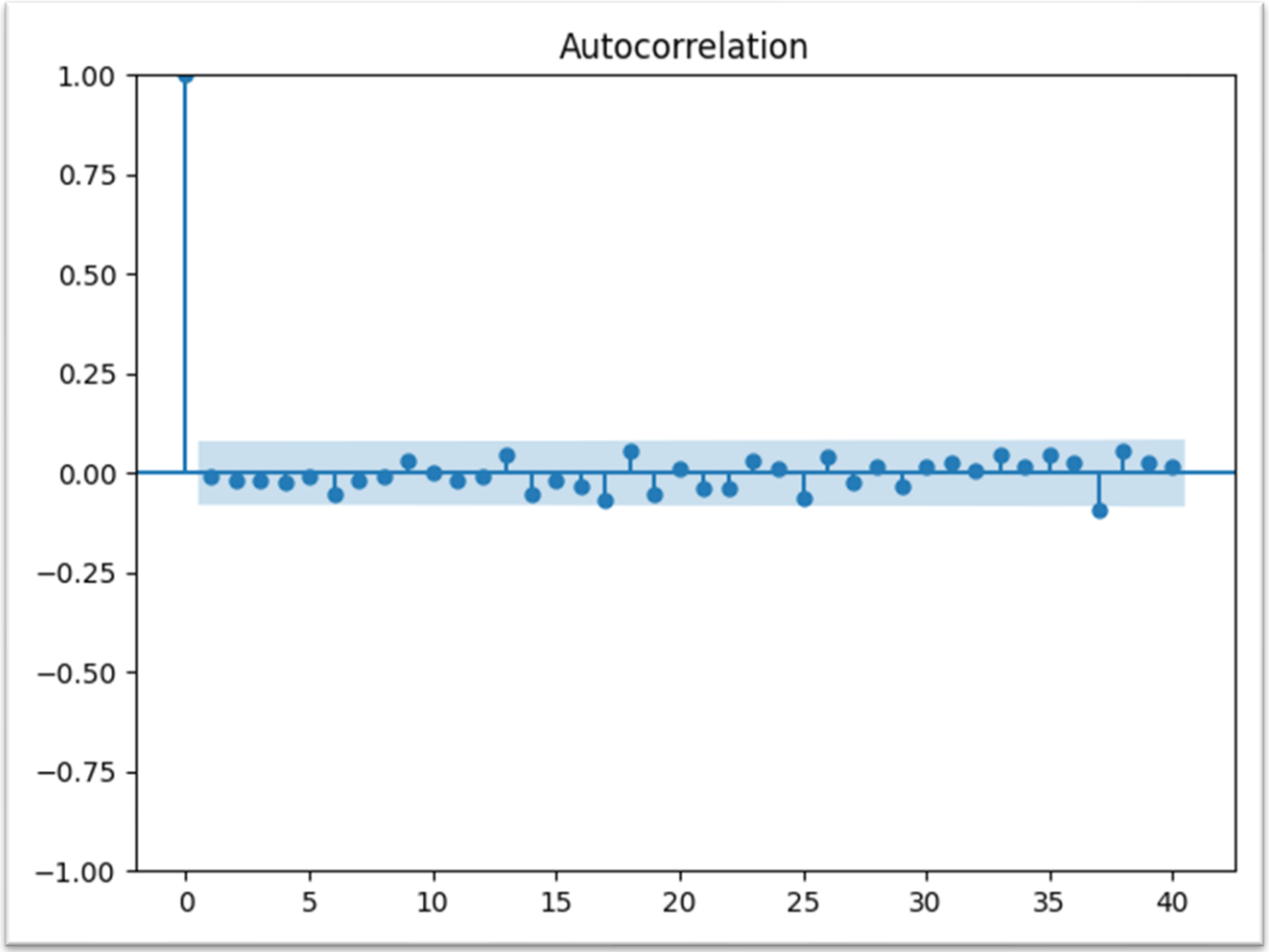
Apresentando o resultado com a melhor ordem e menor BIC.

ARIMA: 6. análise de resíduos

Realize a análise de resíduos e o teste o Ljung-Box:

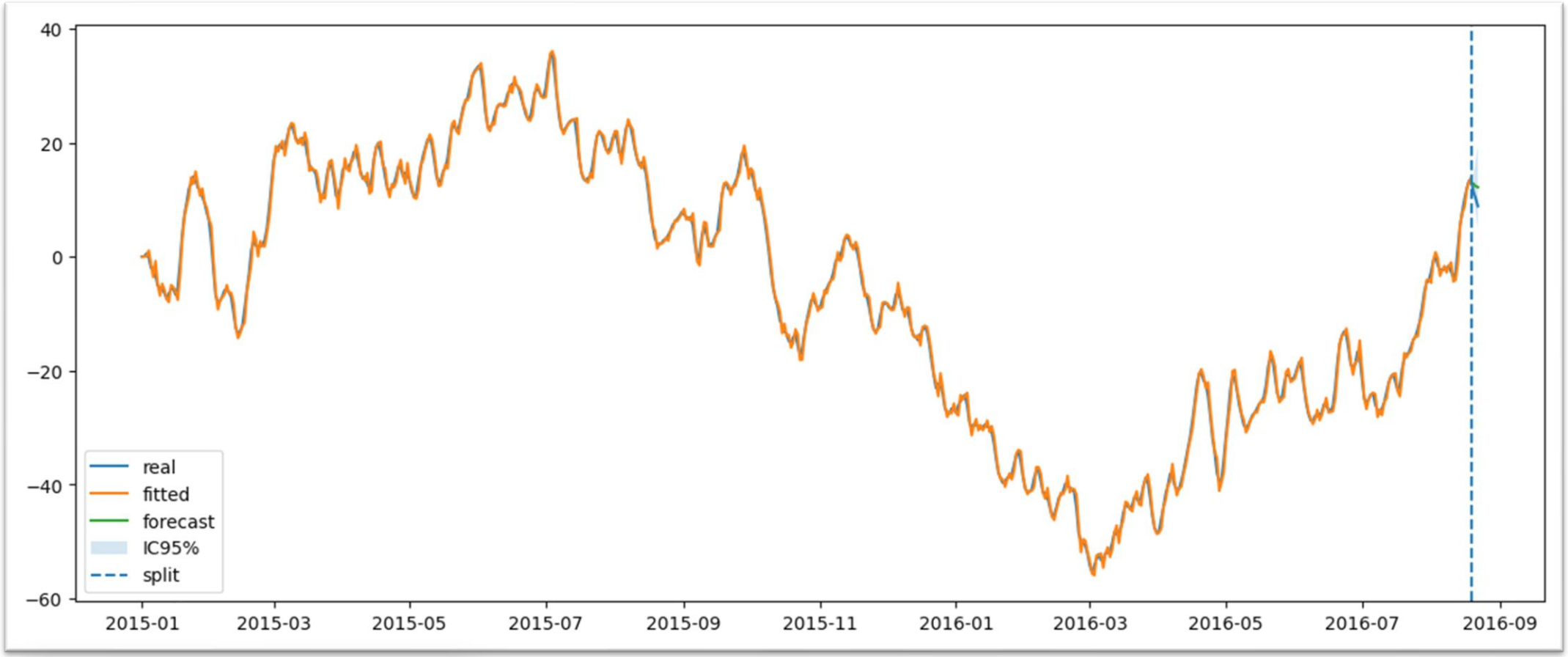
ARIMA: 6. análise de resíduos

Realize a análise de resíduos e o teste o Ljung-Box:



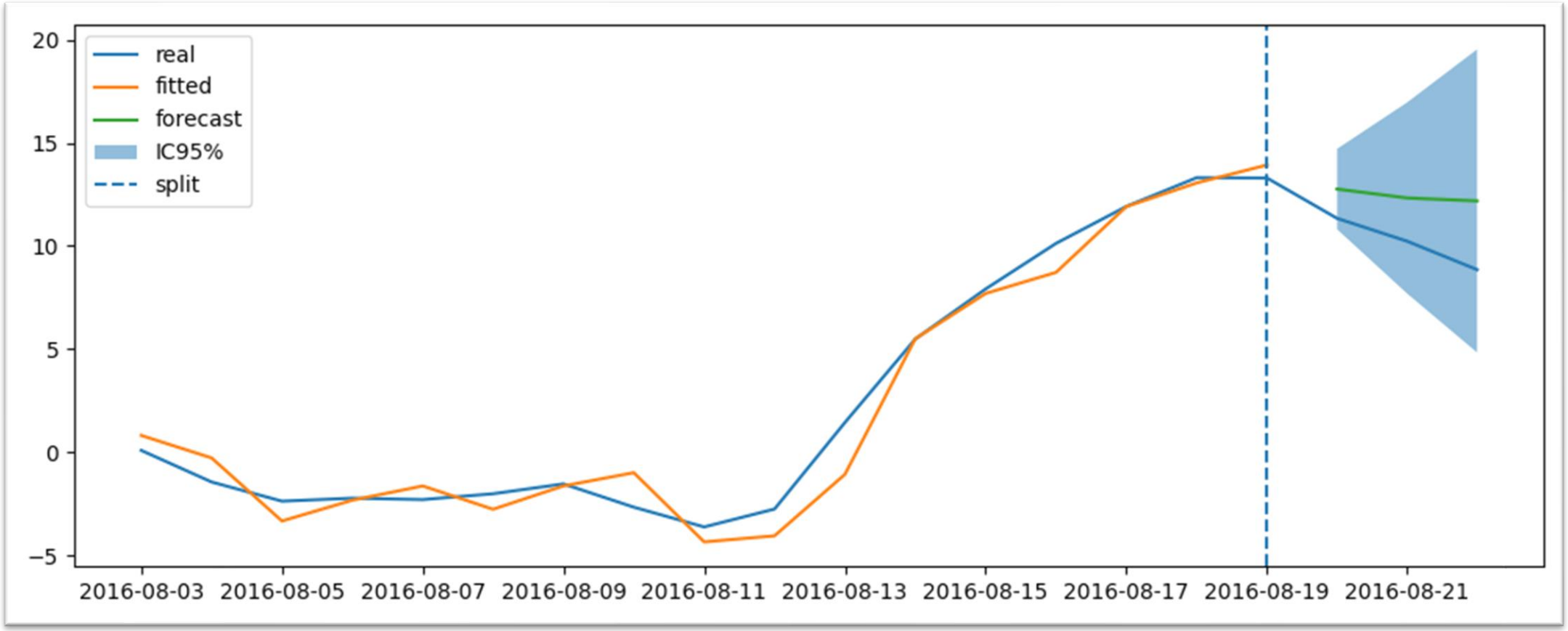
	lb_stat	lb_pvalue
1	0.030210	0.862014
2	0.209577	0.900515
3	0.386811	0.942953
4	0.667427	0.955284
5	0.710148	0.982407
6	2.544041	0.863507
7	2.795183	0.903281
8	2.823750	0.944926
9	3.445915	0.943970
10	3.447323	0.968857
11	3.625562	0.979620
12	3.676406	0.988593
13	5.016826	0.974821
14	6.861075	0.939935
15	7.056867	0.956052
16	7.641273	0.958835
17	10.396502	0.886165
18	12.305359	0.831078
19	13.901541	0.789400
20	13.984023	0.831305

ARIMA: 7. Forecast




```
MAE= 2.285 RMSE= 2.419
```

ARIMA: 7. Forecast




```
MAE= 2.285 RMSE= 2.419
```

ARIMA: 7. Forecast

Realizando a previsão:

```
# previsao (h = len(test))
h = len(test)
fc = melhor_modelo.get_forecast(steps=h)
mean = fc.predicted_mean
ci = fc.conf_int(alpha=0.05)
```

- **define o horizonte** igual ao tamanho do conjunto de teste (nº de passos à frente)
- **h previsões** à frente a partir do fim do treino
- série com os **valores previstos** para cada passo do horizonte
- **intervalos de 95%** (1−α) para a previsão

ARIMA: 7. Forecast

```
fitted = melhor_modelo.fittedvalues
x_ci = ci.index.to_pydatetime()

plt.figure(figsize=(12,5))
plt.plot(df.index, df["y"].values, label="real")
plt.plot(fitted.index, fitted.values, label="fitted")
plt.plot(mean.index, mean.values, label="forecast")
plt.fill_between(x_ci, ci.iloc[:,0].values, ci.iloc[:,1].values, alpha=0.2, label="IC95%")
plt.axvline(train.index[-1], ls="--", label="split")
plt.legend()
plt.tight_layout()
plt.show()
```

ARIMA: 7. Forecast

```
fitted = melhor_modelo.fittedvalues
x_ci = ci.index.to_pydatetime()

plt.figure(figsize=(12,5))
plt.plot(df.index, df["y"].values, label="real")
plt.plot(fitted.index, fitted.values, label="fitted")
plt.plot(mean.index, mean.values, label="forecast")
plt.fill_between(x_ci, ci.iloc[:,0].values, ci.iloc[:,1].values, alpha=0.2, label="IC95%")
plt.axvline(train.index[-1], ls="--", label="split")
plt.legend()
plt.tight_layout()
plt.show()
```

- **valores ajustados no treino (in-sample) já no nível da série**
- **converte o índice do ci para datetimes (paraa o fill_between)**

ARIMA: 7. Forecast

```
fitted = melhor_modelo.fittedvalues
x_ci = ci.index.to_pydatetime()

plt.figure(figsize=(12,5))
plt.plot(df.index, df["y"].values, label="real")
plt.plot(fitted.index, fitted.values, label="fitted")
plt.plot(mean.index, mean.values, label="forecast")
plt.fill_between(x_ci, ci.iloc[:,0].values, ci.iloc[:,1].values, alpha=0.2, label="IC95%")
plt.axvline(train.index[-1], ls="--", label="split")
plt.legend()
plt.tight_layout()
plt.show()
```

- **plota a série real completa.**
- **plota o ajuste in-sample do modelo.**
- **plota a previsão multi-step**

ARIMA: 7. Forecast

```
fitted = melhor_modelo.fittedvalues
x_ci = ci.index.to_pydatetime()

plt.figure(figsize=(12,5))
plt.plot(df.index, df["y"].values, label="real")
plt.plot(fitted.index, fitted.values, label="fitted")
plt.plot(mean.index, mean.values, label="forecast")
plt.fill_between(x_ci, ci.iloc[:,0].values, ci.iloc[:,1].values, alpha=0.2, label="IC95%")
plt.axvline(train.index[-1], ls="--", label="split")
plt.legend()
plt.tight_layout()
plt.show()
```

- **sombreia o intervalo (funil/cone de errro) entre os limites inferior/superior**
- **linha vertical marcando o corte treino/teste**

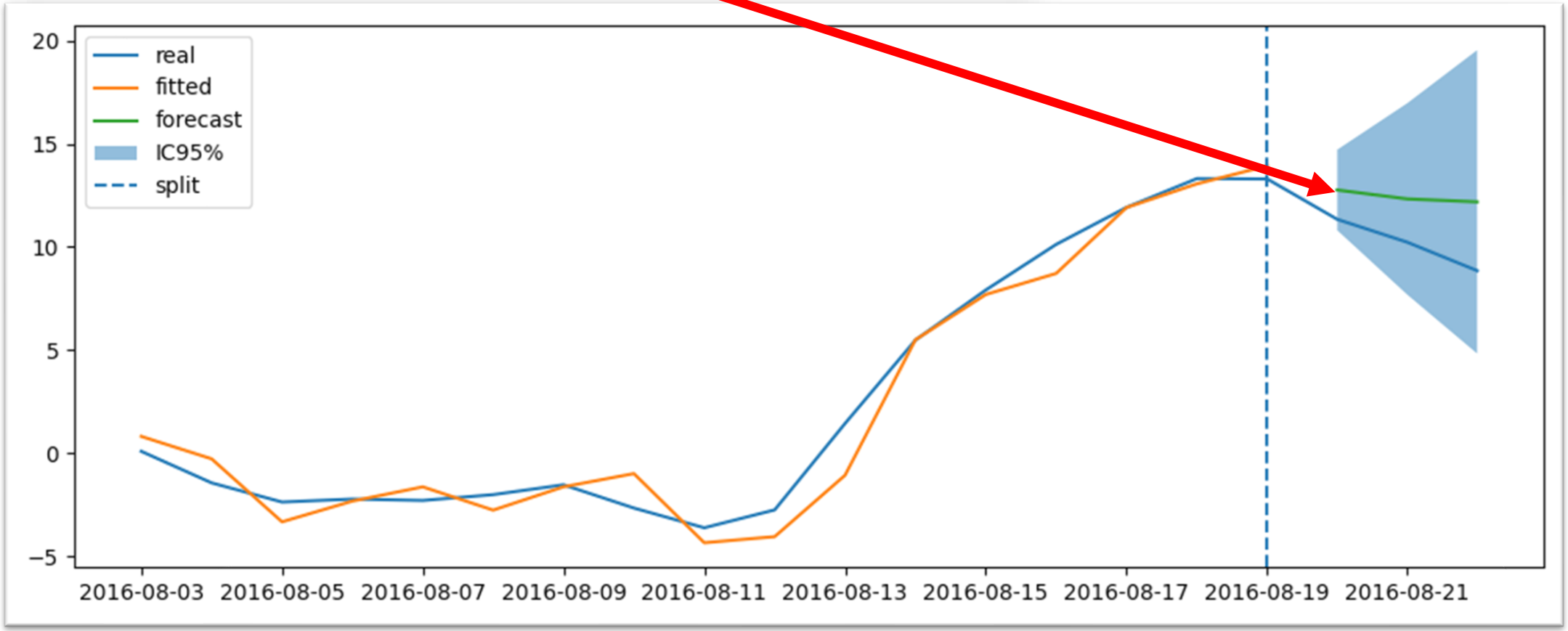
ARIMA: 7. Forecast

```
# resumo das previsoes
print(fc.summary_frame())
```

```
y                mean   mean_se  mean_ci_lower  mean_ci_upper
2016-08-20  12.763159  0.997217      10.808650      14.717667
2016-08-21  12.330442  2.366967       7.691271      16.969613
2016-08-22  12.187644  3.754996       4.827987      19.547301
```

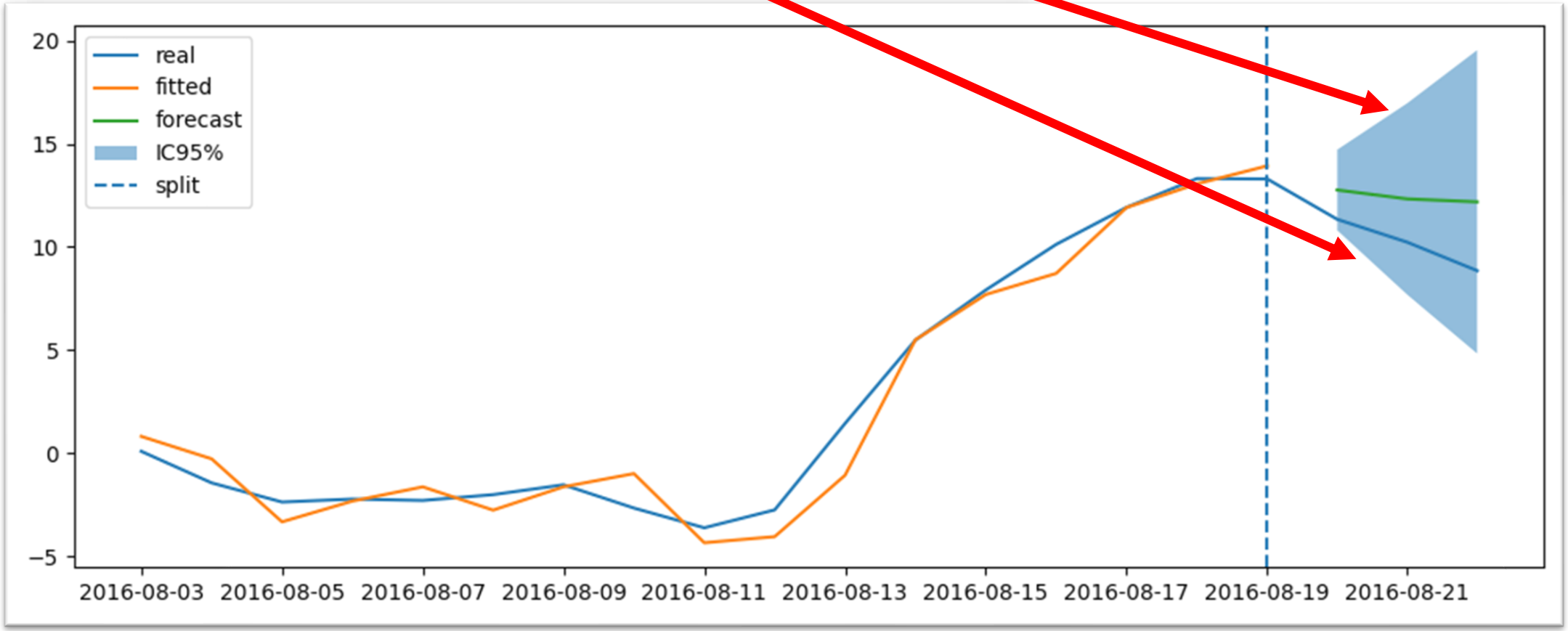
ARIMA: 7. Forecast

y	mean	mean_se	mean_ci_lower	mean_ci_upper
2016-08-20	12.763159	0.997217	10.808650	14.717667
2016-08-21	12.330442	2.366967	7.691271	16.969613
2016-08-22	12.187644	3.754936	4.827987	19.547301



ARIMA: 7. Forecast

y	mean	mean_se	mean_ci_lower	mean_ci_upper
2016-08-20	12.763159	0.997217	10.808650	14.717667
2016-08-21	12.330442	2.366967	7.691271	16.969613
2016-08-22	12.187644	3.754996	4.827987	19.547301



ARIMA: entendendo tudo 😎 😎 😎

Parâmetros (p,d,q)	Modelo	Descrição
ARIMA(p,0,0)	**AR(p)**	Apenas parte auto-regressiva (usa valores passados).
ARIMA(0,0,q)	**MA(q)**	Apenas parte de média móvel (usa erros passados).
ARIMA(p,0,q)	**ARMA(p,q)**	Combinação de AR e MA em série **já estacionária**.
ARIMA(p,d,0)	**AR(dif)** ou AR integrado	Modelo auto-regressivo aplicado após d diferenciações.
ARIMA(0,d,q)	**MA(dif)** ou MA integrado	Modelo de média móvel aplicado após d diferenciações.
ARIMA(p,d,q)	**ARIMA**	Forma geral: diferenciação + AR + MA. Se d>0, lida com séries não estacionárias.

```
Melhor ARIMA(0, 1, 3)  BIC=1715.06
```

ARIMA: entendendo tudo 😎 😎 😎

Parâmetros (p,d,q)	Modelo	Descrição
ARIMA(p,0,0)	**AR(p)**	Apenas parte auto-regressiva (usa valores passados).
ARIMA(0,0,q)	**MA(q)**	Apenas parte de média móvel (usa erros passados).
ARIMA(p,0,q)	**ARMA(p,q)**	Combinação de AR e MA em série **já estacionária**.
ARIMA(p,d,0)	**AR(dif)** ou AR integrado	Modelo auto-regressivo aplicado após d diferenciações.
ARIMA(0,d,q)	**MA(dif)** ou MA integrado	Modelo de média móvel aplicado após d diferenciações.
ARIMA(p,d,q)	**ARIMA**	Forma geral: diferenciação + AR + MA. Se d>0, lida com séries não estacionárias.

```
Melhor ARIMA(0, 1, 3)  BIC=1715.06
```

- **p = 0:** as mudanças não têm memória nos próprios valores passados
- **d = 1:** precisa 1 diferença para ficar estacionária
- **q = 3:** as mudanças dependem erros dos últimos 3 períodos, ou seja, um evento inesperado hoje costuma "ecoar" por até 3 passos e depois se dissipa

Instituto J&F

Centro de Educação para os Negócios